

What Does It Take to Support Problem Solving in Programming Classrooms? A New Framework from the K-12 Teacher Perspective

Ally Limke
Computer Science
North Carolina State University
Raleigh, USA
anlimke@ncsu.edu

Saminur Islam
Computer Science
North Carolina State University
Raleigh, USA
sislam8@ncsu.edu

Bahare Riahi
Computer Science
North Carolina State University
Raleigh, USA
briaahi@ncsu.edu

Xiaoyi Tian
Computer Science
North Carolina State University
Raleigh, USA
xtian9@ufl.edu

Marnie Hill
Computer Science
North Carolina State University
Raleigh, USA
mehill6@ncsu.edu

Veronica Cateté
Computer Science
North Carolina State University
Raleigh, USA
vmcatete@ncsu.edu

Tiffany Barnes
Computer Science
North Carolina State University
Raleigh, USA
tmbarnes@ncsu.edu

Abstract

Developers rarely build programming environments that help secondary teachers support student learning. We interviewed 11 K12 teachers to discover how they support students learning to program and how tools might assist their teaching practice. Based on thematic analysis and organizing teacher activities around student actions, we have derived a new framework that can be used to design a programming learning system to support teachers. Our results suggest that teachers structure their activities based on their ideals about effective programming teaching and learning, and student problem solving and help-seeking processes. Therefore, our framework relates the themes we discovered about teacher activities to ideals and student problem solving in a time-based framework that can inform the design for new programming learning systems.

CCS Concepts

• **Human-centered computing** → **Empirical studies in HCI**; • **Applied computing** → **Computer-assisted instruction**.

Keywords

K12 Teachers, Programming Education, Secondary Computing Education, Orchestrating Computing Activities, Teacher Interviews

ACM Reference Format:

Ally Limke, Saminur Islam, Bahare Riahi, Xiaoyi Tian, Marnie Hill, Veronica Cateté, and Tiffany Barnes. 2025. What Does It Take to Support Problem Solving in Programming Classrooms? A New Framework from the K-12 Teacher Perspective. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '25)*, April 26–May 01, 2025, Yokohama, Japan. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3706599.3719763>

1 Introduction

K-12 Computer science (CS) educational policy initiatives are increasing globally [13]. Most CS curricula include programming, with teachers playing a vital role in supporting student learning [8, 35]. Observing, interpreting, and managing classroom events have been shown to be complex tasks that increase in complexity as teachers gain classroom experience [33]. K-12 computing teachers have reported many challenges including heavy workloads and a lack of confidence [32]. To better support teachers and adapt programming learning systems (PLSs) to their needs, it is essential to understand how programming is facilitated in classrooms.

The crucial role of K-12 teachers in programming learning has been largely neglected or marginalized in curricula, tools, and research, with most teacher-focused CS education research centering around teacher professional development and barriers [5, 25, 26, 29]. Only a few studies have focused on the pedagogical practices of teachers in supporting programming learning [17], especially regarding tool use [28]. In this study, we address this research gap by interviewing eleven secondary teachers (grades 6-12) with varied levels of CS teaching experience to understand the tasks that teachers undertake when leading programming activities in their classroom. Specifically to answer the following research question:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI EA '25, Yokohama, Japan

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1395-8/25/04

<https://doi.org/10.1145/3706599.3719763>

How do grade 6-12 teachers support students in programming activities?

The main contribution of this paper is to identify the activities that teachers undertake to support students in learning to program from the teacher perspective, and to organize them into a framework that facilitates the design of technologies that are more likely to be adopted by teachers. The framework integrates the problems that teachers are trying to solve and the student attributes they feel contribute to those problems, allowing technologies to be designed to address these problems directly.

2 Related Work

K-12 programming activities often include phases like an introduction to spark student interest, sometimes using “unplugged” activities that don’t involve coding [31], followed by a progression of tasks to build familiarity with CS or Computational Thinking (CT) concepts. A common progression is the “Use-Modify-Create” approach, where students use a program, modify it, and eventually create their own [23]. In a 2014 survey of 300 U.K. teachers, Sentance and Csizmadia [30] identified five teacher strategies to support programming instruction: unplugged activities, task contextualization (linking tasks to real-world contexts), collaborative learning (e.g., pair programming), teaching CT concepts [7], and scaffolding tasks (e.g., starter code, hints).

Some work has been done on identifying teacher barriers to leading computing. In a systematic review of the literature on introductory programming from 2003 to 2017, Szabo et al. [32] found that computing teachers faced many barriers to teaching programming curriculum including timetabling, teacher work-load, and a lack of confidence. The article identified that there was no work on widespread initiatives to train and support teachers in teaching programming. Additionally, most introductory programming tools focus on the programming environment, programming practice, or algorithm visualization [22] rather than on supporting teachers in their practices during classroom sessions.

There is a growing number of “integrating teachers” who integrate programming into their non-computing classrooms such as science and math [15, 16], and an increasing number of in-service teachers with limited programming experience teaching computer science classes. Celepkolu et al. [5] found that integrating teachers struggle with limited exposure to and a lack of understanding of CS. Through case studies with three teachers, Barksdale [3] highlights the importance of teacher professional development (PD), “conceptualizing CS/CT for their own classroom, (missing) connections in student-centered, collaborative, and balanced pedagogical strategies; and the (un)plugged fun and supportive role of technology.” In co-design workshops for integrating artificial intelligence (a sub-area of CS), teachers shared that they needed scaffolding in AI tools to lead ethics and data discussions [21]. Integrating teachers encounter many of the same pain points as experienced computing teachers, often without formal computing training and with additional challenges of connecting computing with their subject content, such as science, math and language arts.

Regarding teacher preferences on tool design, we conducted participatory design sessions with three K-12 teachers in which the

teachers produced a list of 14 features they wanted in a Programming Learning System (PLS), like remote screen viewing, autosaving code, and support for differentiated instruction - the practice of adapting curricular materials to address diverse student needs [20]. These insights were echoed in our follow-up survey [19]. While participatory design and surveys capture teachers’ stated preferences, they do not always reveal the practical constraints and workarounds teachers use in real-world settings. This paper builds on these studies by analyzing teacher practices and attitudes, offering design recommendations that better align with how programming learning tools are used in practice.

3 Method

We conducted 45-minute, IRB-approved semi-structured interviews with 11 teachers from the United States to explore their practices and struggles in supporting grades 6-12 students in programming. Teachers were recruited via email and compensated for participating the interviews. Table 1 shows our teacher participants’ demographics and teaching background. Three teachers were integrating programming into other subjects (“integrating teachers”), and eight taught computing. Six teachers taught upper secondary (grades 9-12), and five taught lower secondary (grades 6-8), with varying levels of experience in teaching computing.

The interview consisted of two parts, the first on pedagogical practices and the second on the desired features for software to support these practices. Following Maguire and Delahunt [24]’s method, three researchers applied open-coding thematic analysis on the interview transcripts. Each transcript was independently coded by 2-3 graduate researchers (1 with 4 years of experience assisting teachers with CT integration and engaging them in HCI research, and the other two with 1-2 years of computing education research experience) who then discussed and agreed upon final codes. For this analysis, we excluded codes related to teacher actions outside of the class period, such as lesson planning and grading, as they fell outside the scope of in-classroom practices. However, we retained codes like communicating classroom culture to parents and adapting or creating curriculum, as these aspects were integral to our proposed framework. Finally, the same researchers sorted the codes into themes.

Table 1: Teacher demographics

Teacher	Gender	Years Leading Computing	Grades	Subject Area
Aurora	Female	7	9-12	Computing
Birch	Female	7	9-12	Computing
Clover	Female	7	9-12	Computing
Dune	Male	7	9-12	Computing
Elm	Male	24	9-12	Computing
Forest	Male	31	9-12	Computational Biology
Gale	Female	1	6-8	Animal Science
Holly	Female	7	6-8	Computing
Iris	Female	7	6-8	Science
Jade	Female	8	6-8	Computing
Kodiak	Male	13	6-8	Computing

4 Deriving a Conceptual Model for Teacher Actions While Students Program

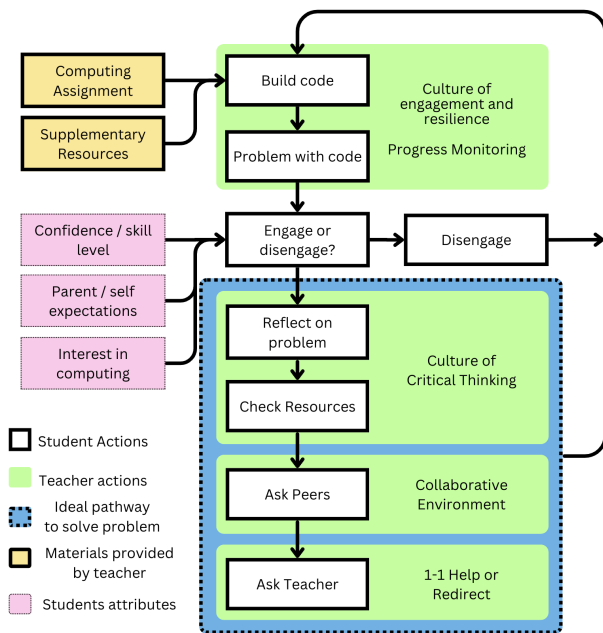


Figure 1: Student actions during programming and teacher created supports

The goal of this study is to derive a model that represents teacher and student actions during programming activities. We involved three experts in computing education to collectively draw the model from the interview themes. Our experts included 1 faculty in computer science with 12 years of experience leading teacher professional development (PD), 1 graduate researcher with 4 years experience in HCI research and assisting teachers with CT integration, and one secondary teacher with 8 years of experience leading teacher PD for CS. Our team reviewed the themes for the actions teachers took to support students while programming in the classroom. The team collaboratively and iteratively organized the themes into groups until a consensus was reached. The themes and groups are listed below, with a visualized framework presented in Fig. 1. The numbers represent the number of participants who mentioned each theme:

Supporting problem-solving and engagement:

Helping students 1-1 (11); fostering programming culture through normalizing mistakes, encouraging collaboration and participation, and inviting creativity (10); teaching critical thinking and self-sufficiency (10); monitoring student engagement and progress (10); organizing peer tutors and partnered work (9); struggles to help all students during class (7); motivating students to ask for help (4); preparing students for college and careers (4); facilitating student presentations (2); providing forgotten login credentials (1); reminding students to save code (1)

Influencing or accommodating student attributes: Considering student interest/choice in activities (8); motivating students

to value programming (4); differentiating activities in real-time (4); communicating classroom culture to parents (3)

Creating and distributing materials to students: Adapts or creates curriculum (10); Provides supplemental resources to students (6)

To create our model, we arranged the themes as a proposed new framework to explain the activities teachers undertake to support programming in the classroom. Initially, the teacher activities all seemed as diverse as the teachers. However, throughout our process of grouping the themes, and in reviewing the interview transcripts, we observed that all of our teachers provided reasoning for each choice about their classroom and student supports rooted in (1) creating/modifying their own curriculum and supporting materials, (2) handling student attributes such as interest, and student and parent attitudes about computing and perceptions about how learning should take place, and (3) the fact that programming involves students interacting with a programming environment (writing code), encountering failure, and (4) committing to providing individual 1-1 help when all else failed. This led to a natural arrangement of the themes as they related to a time-based sequence of **student actions** to write code until reaching an impasse/encountering a problem, and then seeking help to fix that problem. The sources of help that teachers provided were organized based on the locus of control of that source (found in the **Ideal pathway** node) - with the first source of help being a student's own ability to think critically, then teacher-provided resources, then peers within the class, and then the teacher themselves. This is in line with the common themes from teachers that students should learn a help-seeking process that students could use into the future when the teachers would not be available (teaching critical thinking and self-sufficiency).

5 Results: What Teachers Do To Orchestrate Computing Activities During Class

We detail the tasks that teachers undertake and the structures that they aim to build during a class, which correspond to the "Teacher actions" boxes in Figure 1. This includes monitoring student engagement and progress, building a classroom culture around resilience, collaboration, participation, creativity, help-seeking, and critical thinking, and helping students, one-on-one, to solve problems.

5.1 Build a Culture of Engagement and Resilience

Teachers actively build a culture of engagement and resilience through reflection and creating relevance to students. Elm asks his students “If you just do it using ChatGPT, or Bard... Why are you in here taking this course?” to help motivate students to actively engage. Other teachers showcase computing career opportunities, host industry speakers to build “relationships with [companies] like Apple or Lenovo” (Clover), and teach “marketable skills that they can take outside of the classroom” (Birch). Others encourage students to “explore passion projects” that “connect to the competencies” (Clover), or challenge students who finish early to reach stretch goals (Gale, Holly). These diverse approaches reflect a variety of motivation and relevance factors.

Teachers build resilience by normalizing struggle and setting expectations for independent problem-solving skills rather than producing correct answers. Birch models struggle by asking her class to help solve problems in her code, stating “At first it was difficult to say, ‘I don’t know’, but then I learned to say ‘Okay, I can’t know everything’. And, so, I will say, ‘let’s work it out together’”. Holly stresses that “mistakes are okay” and feels “It’s better that [students] are challenged, before [college]”. Dune teaches that “taking risks and failing and doing things that are hard can be extremely productive,” and the key to success is “perseverance.” A focus on resilience is reflected in assessments, with four teachers allowing students to correct work to learn from mistakes. Three teachers help set parent expectations about the importance of struggle as a natural part of computing. Elm shared, “I think sometimes the parents can [think], ‘Oh, I, you know, my child has to make all A’s, all A pluses’... And you know, computer science is hard.” He tells parents “I know you want [your child] to be successful... I encourage you to let them sit and stew [in challenges] and see what happens.”

5.1.1 Progress Monitoring. 10 of the 11 teachers monitor progress and help keep students engaged by walking around and viewing student screens (9) or viewing their progress through a PLS (1). Six teachers want to monitor progress through “real-time [data on] how students are performing” (Iris) for overall pacing, and alerts when students are “stuck on something” (Iris) so they can scaffold problem solving (see Fig. 1 in blue). Dune “can look right at [student code] and see ... their areas [of struggle].” Gale asks students about their progress, helping them think critically and learn to self-monitor. Gale and Jade also view screens remotely; “I can see all of their screens at once... like who’s blank, who’s typing, who’s erasing. And it helps me know where I need to go [to provide help]” (Gale). Importantly, **every** teacher reported that they provide one-on-one student support, but since they cannot help every student during class, they build many supports around the problem-solving process (see Fig. 1) to help students become more independent and resilient problem solvers.

5.2 Finding a solution: Building a Culture of Critical Thinking

To help students solve problems, 10 of our teachers foster independence, critical thinking, and appropriate help-seeking. Since “some kids are gonna be a little bit too nervous to, like, raise their hands” (Gale), teachers encourage asking for help. However, some teachers remind students of their responsibility for learning, with Jade telling them, “I’m always available [for] questions, but I’m not going to do it for you” and Forest stating if “it’s a problem they should be able to fix, I’ll tell them”. Four teachers model critical thinking skills by asking students questions to help learn what questions they can ask themselves to solve their own problems. Holly explained, “I don’t give them answers... I like playing stupid. And I’m like, ‘I have no idea. I don’t know. Let’s figure it out.’” Similarly, Aurora shared, “If they ask a question, I’ll ask them three more questions to get them thinking.” Three teachers want students to learn independently, e.g. through supplemental resources. Dune shows students “how they can find the answer,” and Elm states “I want the kids to teach themselves... providing them the

resources, and then learning how to get out of their way instead of being a fountain of knowledge”. In Fig. 1, we include providing materials and critical thinking questions as teacher activities to promote student-centered critical thinking skills in the problem solving process.

5.3 Second Line of Defense: Building a Collaborative Environment

Nine teachers build a collaborative environment to help students build important skills while learning from one another. Some teachers pair students so “they have to learn how to communicate” (Holly), including peer review “glows and grows,” something their partner did well (glows), and something they can improve on (grows) (Clover). Six teachers set classroom expectations to help one another, telling students “We’re all a family, and ... we have to support one another” (Jade). Gale rewards students for giving or accepting help from peers. Elm and Iris pair less-confident students with more confident students, with the goal of more confident students building skills in providing help and less confidence students getting peer support. But Holly shared that effective one-on-one peer tutoring can be difficult, “What I find the most challenging is... how to get students to help them like, as peers, but not touch their mouse... and do it for them”. Three teachers leverage the large differences in student prior experience to ask more advanced students to present to the class. Students who are “super far ahead” (Kodiak) enjoy teaching the class because “they always want to show what they know” (Birch).

5.4 Still need help? Teacher-Student Instructional Interventions

Some teachers encourage students to reference resources, use critical thinking, and ask their peers for help because, as Aurora shared, “there’s one of me in a classroom full of kids. I can only help so many people”. When students have exhausted their resources, teachers provide one-on-one instructional support. Three teachers (Birch, Forest, Gale) provide extra resources, like answer keys and additional notes. Jade uses screen monitoring paired with messages: “If they’re stuck on one thing, you know I can send them a little instant message.” If Iris notices a student is struggling, she checks in with that student often so “that person does not feel ... left out of the whole coding process.” The time teachers have for instructional interventions is already limited, and it’s further reduced by the need to address management tasks caused by student disorganization, such as recovering lost passwords or ensuring code is saved properly (Gale, Iris).

5.5 Design Implications

In this section, we provide design recommendations for PLS designs to empower teachers based on teacher interview themes, organized in Table 2.

We recommend that PLSs showcase integrated materials’ relevance to teachers, parents, and students. A meta-analysis by Jeynes found that parental expectations have a statistically significant impact on students’ academic achievement [14]. Generating shareable

Table 2: Themes and corresponding design considerations, Letters indicate the first initials of participants’ pseudonyms

Teacher Practices	Design Considerations
- Foster programming culture: normalize mistakes, encourage collaboration and participation, invite creativity (A,B,C,D,E,F,G,H,I,J,K)	Tools to help teachers reflect on their desires for and establish and share a classroom culture
- Motivate students to value programming (G,H,I,J)	
- Students present their work to the class (B,J)	
- Prepare students for college and careers (B,C,F,H)	Showcasing integrated material relevance to teachers, parents, and students (career, college, real world)
- Communicate culture to parents (C,E,F)	
- Organize peer tutors and partnered work (A,B,C,E,G,H,I,J,K)	Orchestrate collaboration and provide students multiple routes for support outside of teacher interventions
- Teach critical thinking / self-sufficiency (A,C,D,E,F,G,H,I,J,K)	
- Monitor student engagement and progress (A,B,C,D,E,G,H,I,J,K)	Tools for teachers to notice need for and initiate informed and inclusive interventions
- Motivate students to ask for help (E,F,G,J)	
- Differentiate activities in real-time (B,F,G,H)	
- Provide forgotten login credentials (G)	Automate classroom management activities in accordance with school privacy and security policies
- Remind students to save code (I)	

links to lessons may support teachers in communicating with parents during (for understanding what their children are doing in class), and after each course (for recruiting future students).

We also suggest tools for orchestrating collaboration and providing multiple support routes outside of teacher interventions. Researchers have investigated teacher dashboards to show which students might need help/intervention during programming [6, 9, 10]. In a PLS this may be integrated through class-wide, grouped, or private help requests. Enabling class-wide help requests viewable by the entire class could support student collaboration. Group-wise help requests, viewable by a subset of the class, could be leveraged when teachers want to support small peer group learning. Private requests to the teacher could help students who are reluctant to ask for help (requested by two teachers). To foster critical thinking skills, the PLS could prompt students requesting help to answer a few questions about their problem. The questions might mimic the classic Rubber Duck Debugging intervention [1] that is commonly used in computer science classrooms to help students learn to debug code by explaining their problem to a rubber duck. Alternatively, a PLS could employ LLM-based Programming Assistants for individual student requests[18].

Collaboration tools can group students based on teacher criteria (i.e., pairing advanced students with struggling students). Similar to our results, past research shows that computing teachers strive to establish a culture of collaboration, but need more strategies and tools to support these efforts [30]. A dashboard that matched low-performing students with high-performing peer tutors was found to increase the amount of help students received and the consistency of help availability [6]. Additionally, it may be useful to integrate collaborative environments, like Netsblox [4], which allow students to concurrently edit the same code. To help teachers hold students accountable for participation while in groups, we recommend allowing teachers to view student edit histories.

We also identified a need for tools to help teachers notice the need for and initiate informed and inclusive interventions, such as integrated screen monitoring. In a earlier survey we conducted with 39 K-12 computing teachers, screen monitoring software was identified as the highest-need feature for a PLS [19]. In this study, five of eleven teachers used screen monitoring or wanted this as a PLS feature. The teachers in this study also often sent additional files or messages to help individual students. Doing so may help students stay organized by keeping all shared resources on the PLS, and prevent distraction with other platforms.

Our teachers also requested real-time data on student performance and struggle, suggesting that dashboards for student data visualizations may be helpful. Past studies show that STEM teachers and students benefit from real-time analytics about student engagement/ idleness [2, 12], number of students who mastered each skill [34], the number of students who made certain errors [12, 34], and which students are experiencing unproductive persistence [12]. At the undergraduate level, there have been efforts to produce classroom-level analytics about student programs to drive interventions [11, 36]. In this study, teachers reported constantly monitoring student progress and off-task behaviors. A PLS could replace this teacher task by detecting and alerting the teacher if a student has been idle for the last few minutes. Automating classroom management activities, like saving student code and resetting passwords, can also free teachers’ time for one-on-one interventions.

6 Limitations & Future Work

In a future survey, we plan to ask a broader sample of teachers how our model fits or diverges from their perceptions of student problem solving in programming and how they support student problem solving. We want to do the same for the participants of this study. This will ensure that we are accurately interpreting the

teacher interview responses and identify how the model should be changed to represent a more diverse sample of teachers.

One limitation of this study was that we relied on teachers to relay their experiences rather than observing them in the classroom. While our results give us insights into a subset of processes they undertake, teachers may not be able to fully recall everything that they do in the classroom. The model, as it is now, represents teacher perceptions of their actions in relation to their students' actions and attributes; however, classroom observations can illuminate how the created model relates to the realities of classrooms.

Another limitation of the work was that we did not have enough participants to differentiate the needs between integrating and computing teachers. When designing for these subpopulations, it may be helpful to know how their needs differ. Future work should investigate K-12 teacher pedagogical practices to support programming, and how those practices change at different points in the school year, and for long- or short-term programming activities.

While there has been some work conducted on student perceptions of the use of their data to provide insights to instructors in smart classrooms [27], future work should also investigate how our design recommendations are viewed by students in terms of their privacy and autonomy. Additionally, other stakeholders should be included in the design process, like parents, administrators, and IT professionals, to lower the barriers of tool adoption.

7 Conclusion

Through thematic analysis of 11 semi-structured interviews of secondary teachers in grades 6-12 classrooms, including both computing teachers as well as non-computing teachers, we found that teachers engage in similar practices when leading programming in their classrooms. However, unique challenges arise in the support of programming learning, namely vast differences in student prior experiences and interests, resilience, critical thinking, collaboration, and help seeking. These challenges mean that teachers are the single most important resource in a classroom, who must build an engaging and supportive culture that promotes resilience, critical thinking, resourcefulness, collaboration, and appropriate help-seeking, while they also strive to provide one-on-one teacher-student support for many students in short class times. As one teacher stated, "There was one of me and 28 of them in a class at a time." Teachers take on the extra burdens that interactive learning environments may be designed to handle (such as individual feedback), those that many programming environments do not yet handle (such as automated assessment), and those that the social nature of learning demands (such as working with peers). This paper contributes a new teacher-elaborated model of student programming problem solving and what teachers do to support it and recommendations for future Programming Learning Systems.

Acknowledgments

This material is based upon work supported by the National Science Foundation under Grants No. 2112635, 2405854, and 2055528. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

References

- [1] Hunt Andrew and Thomas David. 2000. The pragmatic programmer: From journeyman to master.
- [2] Sinem Aslan, Nese Alyuz, Cagri Tanriover, Sinem E. Mete, Eda Okur, Sidney K. D'Mello, and Asli Arslan Esmé. 2019. Investigating the Impact of a Real-time, Multimodal Student Engagement Analytics Technology in Authentic Classrooms. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland Uk) (CHI '19). Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/3290605.3300534>
- [3] Sarah Barksdale. 2023. "It's Hard Work"—K-12 Teachers' Enacted Curriculum of Computational Thinking and Computer Science Integration. Ph. D. Dissertation. University of Minnesota.
- [4] Brian Broll, Akos Lédeczi, Peter Volgyesi, Janos Sallai, Miklos Maroti, Alexia Carrillo, Stephanie L. Weeden-Wright, Chris Vanags, Joshua D. Swartz, and Melvin Lu. 2017. A Visual Programming Environment for Learning Distributed Programming. In *Proceedings of the 2017 ACM SIGCSE Technical Symposium on Computer Science Education* (Seattle, Washington, USA) (SIGCSE '17). Association for Computing Machinery, New York, NY, USA, 81–86. <https://doi.org/10.1145/3017680.3017741>
- [5] Mehmet Celepkolu, Erin O'Halloran, and Kristy Elizabeth Boyer. 2020. Upper Elementary and Middle Grade Teachers' Perceptions, Concerns, and Goals for Integrating CS into Classrooms. In *Proceedings of the 51st ACM Technical Symposium on Computer Science Education* (Portland, OR, USA) (SIGCSE '20). Association for Computing Machinery, New York, NY, USA, 965–970. <https://doi.org/10.1145/3328778.3366937>
- [6] Nicholas Diana, Michael Eagle, John Stamper, Shuchi Grover, Marie Bienkowski, and Satabdi Basu. 2017. An instructor dashboard for real-time analytics in interactive programming assignments. In *Proceedings of the Seventh International Learning Analytics & Knowledge Conference* (Vancouver, British Columbia, Canada) (LAK '17). Association for Computing Machinery, New York, NY, USA, 272–279. <https://doi.org/10.1145/3027385.3027441>
- [7] Yihuan Dong, Veronica Catete, Robin Jocius, Nicholas Lytle, Tiffany Barnes, Jennifer Albert, Deepti Joshi, Richard Robinson, and Ashley Andrews. 2019. PRADA: A Practical Model for Integrating Computational Thinking in K-12 Education. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education* (Minneapolis, MN, USA) (SIGCSE '19). Association for Computing Machinery, New York, NY, USA, 906–912. <https://doi.org/10.1145/3287324.3287431>
- [8] Wendy DuBow, Alexis Kaminsky, and Joanna Weidler-Lewis. 2017. Multiple factors converge to influence women's persistence in computing: A qualitative analysis. *Computing in Science & Engineering* 19, 3 (2017), 30–39.
- [9] Qian Fu, Xue Bai, Yafeng Zheng, Runsheng Du, Dongqing Wang, and Tianyi Zhang. 2023. VisOJ: Real-time visual learning analytics dashboard for online programming judge. *The Visual Computer* 39, 6 (2023), 2393–2405.
- [10] Xinyu Fu, Atsushi Shimada, Hiroaki Ogata, Yuta Taniguchi, and Daiki Suehiro. 2017. Real-time learning analytics for C programming language courses. In *Proceedings of the Seventh International Learning Analytics & Knowledge Conference* (Vancouver, British Columbia, Canada) (LAK '17). Association for Computing Machinery, New York, NY, USA, 280–288. <https://doi.org/10.1145/3027385.3027407>
- [11] Elena L. Glassman, Jeremy Scott, Rishabh Singh, Philip J. Guo, and Robert C. Miller. 2015. OverCode: Visualizing variation in student solutions to programming problems at scale. *ACM Transactions on Computer-Human Interaction (TOCHI)* 22, 2 (2015), 1–35.
- [12] Kenneth Holstein, Bruce M. McLaren, and Vincent Alevén. 2018. Student Learning Benefits of a Mixed-Reality Teacher Awareness Tool in AI-Enhanced Classrooms. In *Artificial Intelligence in Education*, Carolyn Penstein Rosé, Roberto Martínez-Maldonado, H. Ulrich Hoppe, Rose Luckin, Manolis Mavrikis, Kaska Porayska-Pomsta, Bruce McLaren, and Benedict du Boulay (Eds.). Springer International Publishing, Cham, 154–168.
- [13] Yu-Chang Hsu, Natalie Roote Irie, and Yu-Hui Ching. 2019. Computational thinking educational policy initiatives (CTEPI) across the globe. *TechTrends* 63 (2019), 260–270.
- [14] William H. Jeynes. 2024. A meta-analysis: The relationship between the parental expectations component of parental involvement with students' academic achievement. *Urban Education* 59, 1 (2024), 63–95.
- [15] Robin Jocius, Jennifer Albert, W. Ian O'Byrne, Deepti Joshi, Richard Robinson, and Melanie Blanton. 2024. Computational thinking infusion as transformative teaching: investigating content area teacher perspectives and practices. *Computer Science Education* 34, 2 (2024), 222–251.
- [16] Robin Jocius, Deepti Joshi, Yihuan Dong, Richard Robinson, Veronica Cateté, Tiffany Barnes, Jennifer Albert, Ashley Andrews, and Nicholas Lytle. 2020. Code, Connect, Create: The 3C Professional Development Model to Support Computational Thinking Infusion. In *Proceedings of the 51st ACM Technical Symposium on Computer Science Education* (Portland, OR, USA) (SIGCSE '20). Association for Computing Machinery, New York, NY, USA, 971–977. <https://doi.org/10.1145/3328778.3366797>
- [17] Celal Murat Kandemir, Filiz Kalelioğlu, and Yasemin Gülbahar. 2021. Pedagogy of Teaching Introductory Text-based Programming in Terms of Computational

- Thinking Concepts and Practices. *Computer Applications in Engineering Education* 29, 1 (2021), 29–45.
- [18] Majeed Kazemitabaar, Runlong Ye, Xiaoning Wang, Austin Zachary Henley, Paul Denny, Michelle Craig, and Tovi Grossman. 2024. CodeAid: Evaluating a Classroom Deployment of an LLM-based Programming Assistant that Balances Student and Educator Needs. In *Proceedings of the CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI '24). Association for Computing Machinery, New York, NY, USA, Article 650, 20 pages. <https://doi.org/10.1145/3613904.3642773>
- [19] Ally Limke, Marnie Hill, Veronica Cateté, and Tiffany Barnes. 2024. A Survey of K-12 Teacher Needs for an Online Programming Learning System. In *Extended Abstracts of the 2024 CHI Conference on Human Factors in Computing Systems* (CHI EA '24). Association for Computing Machinery, New York, NY, USA, Article 23, 7 pages. <https://doi.org/10.1145/3613905.3651110>
- [20] Ally Limke, Nicholas Lytle, Sana Mahmoud, Maggie Lin, Marnie Hill, Veronica Cateté, and Tiffany Barnes. 2023. Participatory Design with Teachers for Block-Based Learning with SnapClass. In *2023 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE, Washington, DC, USA, 173–178. <https://doi.org/10.1109/VL-HCC57772.2023.00028>
- [21] Phoebe Lin and Jessica Van Brummelen. 2021. Engaging Teachers to Co-Design Integrated AI Curriculum for K-12 Classrooms. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems* (Yokohama, Japan) (CHI '21). Association for Computing Machinery, New York, NY, USA, Article 239, 12 pages. <https://doi.org/10.1145/3411764.3445377>
- [22] Andrew Luxton-Reilly, Simon, Ibrahim Albluwi, Brett A. Becker, Michail Giannakos, Amruth N. Kumar, Linda Ott, James Paterson, Michael James Scott, Judy Sheard, and Claudia Szabo. 2018. Introductory programming: a systematic literature review. In *Proceedings Companion of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education* (Larnaca, Cyprus) (ITiCSE 2018 Companion). Association for Computing Machinery, New York, NY, USA, 55–106. <https://doi.org/10.1145/3293881.3295779>
- [23] Nicholas Lytle, Veronica Cateté, Danielle Boulden, Yihuan Dong, Jennifer Houchins, Alexandra Milliken, Amy Isvik, Dolly Bounajim, Eric Wiebe, and Tiffany Barnes. 2019. Use, Modify, Create: Comparing Computational Thinking Lesson Progressions for STEM Classes. In *Proceedings of the 2019 ACM Conference on Innovation and Technology in Computer Science Education* (Aberdeen, Scotland UK) (ITiCSE '19). Association for Computing Machinery, New York, NY, USA, 395–401. <https://doi.org/10.1145/3304221.3319786>
- [24] Moira Maguire and Brid Delahunt. 2017. Doing a thematic analysis: A practical, step-by-step guide for learning and teaching scholars. *All Ireland journal of higher education* 9, 3 (2017), 1–14.
- [25] Maria Cecilia Martinez, Marcos J. Gomez, Marco Moresi, and Luciana Benotti. 2016. Lessons Learned on Computer Science Teachers Professional Development. In *Proceedings of the 2016 ACM Conference on Innovation and Technology in Computer Science Education* (Arequipa, Peru) (ITiCSE '16). Association for Computing Machinery, New York, NY, USA, 77–82. <https://doi.org/10.1145/2899415.2899460>
- [26] Alexandra Milliken, Christa Cody, Veronica Catete, and Tiffany Barnes. 2019. Effective Computer Science Teacher Professional Development: Beauty and Joy of Computing 2018. In *Proceedings of the 2019 ACM Conference on Innovation and Technology in Computer Science Education* (Aberdeen, Scotland UK) (ITiCSE '19). Association for Computing Machinery, New York, NY, USA, 271–277. <https://doi.org/10.1145/3304221.3319779>
- [27] Tricia J. Ngoon, David Kovalev, Prasoon Patidar, Chris Harrison, Yuvraj Agarwal, John Zimmerman, and Amy Ogan. 2023. "An Instructor is [already] able to keep track of 30 students": Students' Perceptions of Smart Classrooms for Improving Teaching & Their Emergent Understandings of Teaching and Learning. In *Proceedings of the 2023 ACM Designing Interactive Systems Conference* (Pittsburgh, PA, USA) (DIS '23). Association for Computing Machinery, New York, NY, USA, 1277–1292. <https://doi.org/10.1145/3563657.3596079>
- [28] Solomon Sunday Oyelere, Friday Joseph Agbo, and Ismaila Temitayo Sanusi. 2022. Developing a Pedagogical Evaluation Framework for Computational Thinking Supporting Technologies and Tools. In *Frontiers in Education*, Vol. 7. Frontiers Media SA, Switzerland, 957739.
- [29] Thomas W. Price, Veronica Cateté, Jennifer Albert, Tiffany Barnes, and Daniel D. Garcia. 2016. Lessons Learned from "BJC" CS Principles Professional Development. In *Proceedings of the 47th ACM Technical Symposium on Computing Science Education* (Memphis, Tennessee, USA) (SIGCSE '16). Association for Computing Machinery, New York, NY, USA, 467–472. <https://doi.org/10.1145/2839509.2844625>
- [30] Sue Sentance and Andrew Csizmadia. 2017. Computing in the curriculum: Challenges and strategies from a teacher's perspective. *Education and information technologies* 22 (2017), 469–495.
- [31] Yukyong Song, Xiaoyi Tian, Nandika Regatti, Gloria Ashiya Katuka, Kristy Elizabeth Boyer, and Maya Israel. 2024. Artificial Intelligence Unplugged: Designing Unplugged Activities for a Conversational AI Summer Camp. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1* (Portland, OR, USA) (SIGCSE 2024). Association for Computing Machinery, New York, NY, USA, 1272–1278. <https://doi.org/10.1145/3626252.3630783>
- [32] Claudia Szabo, Judy Sheard, Andrew Luxton-Reilly, Simon, Brett A. Becker, and Linda Ott. 2019. Fifteen Years of Introductory Programming in Schools: A Global Overview of K-12 Initiatives. In *Proceedings of the 19th Koli Calling International Conference on Computing Education Research* (Koli, Finland) (Koli Calling '19). Association for Computing Machinery, New York, NY, USA, Article 8, 9 pages. <https://doi.org/10.1145/3364510.3364513>
- [33] Charlotte E Wolff, Halszka Jarodzka, and Henny PA Boshuizen. 2021. Classroom management scripts: A theoretical model contrasting expert and novice teachers' knowledge and awareness of classroom events. *Educational Psychology Review* 33, 1 (2021), 131–148.
- [34] Franceska Khakaj, Vincent Aleven, and Bruce M. McLaren. 2017. Effects of a Teacher Dashboard for an Intelligent Tutoring System on Teacher Knowledge, Lesson Planning, Lessons and Student Learning. In *Data Driven Approaches in Digital Education*, Élise Lavoué, Hendrik Drachsler, Katrien Verbert, Julien Broisin, and Mar Pérez-Sanagustín (Eds.). Springer International Publishing, Cham, 315–329.
- [35] Enwei Xu, Wei Wang, and Qingxia Wang. 2023. A meta-analysis of the effectiveness of programming teaching in promoting K-12 students' computational thinking. *Education and Information Technologies* 28, 6 (2023), 6619–6644.
- [36] Ashley Ge Zhang, Yan Chen, and Steve Oney. 2023. VizProg: Identifying Misunderstandings By Visualizing Students' Coding Progress. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) (CHI '23). Association for Computing Machinery, New York, NY, USA, Article 596, 16 pages. <https://doi.org/10.1145/3544548.3581516>

Received 23 January 2025